

Statistical Arbitrage (Pairs Trading) Strategy

Theory, mathematical modeling and production execution architecture

Author	Quantitative Trading Architecture
Asset Universe	Technology & Semiconductor Stocks / Crypto & Forex Crosses
Execution Timeframe	Daily (1D), 4-Hour (4h), 1-Hour (1h), or 15-Minute (15m)

Executive Summary

This document details the full theoretical foundation, mathematical modeling, and production execution architecture for a **Dynamic Statistical Arbitrage (Pairs Trading) Strategy**. The strategy identifies statistically bound pairs of assets, models their equilibrium price relationship dynamically, and trades short-term mean-reversion spreads when prices temporarily diverge from equilibrium.

Component	Method	Role
Cointegration Screening	Engle-Granger, $p < 0.05$	Filters 190 combinations down to ~32 cointegrated pairs
Dynamic State Modeling	Kalman Filter updates	Estimates β_t and α_t on every bar
Mean-Reversion Engine	Ornstein-Uhlenbeck (OU)	Calculates half-life (λ) and Z-score signals
Execution & Risk Controls	7-day re-screening	Entries at $ Z \geq 2.0$, exits at $Z = 0$, stop-loss at 3.5

1. Cointegration & Pair Selection

Correlation measures linear directional movement, whereas cointegration measures long-term statistical equilibrium. Two non-stationary price series $P_{1,t}$ and $P_{2,t}$ are cointegrated if a linear combination of them yields a stationary (I(0)) residual spread series S_t .

Engle-Granger Two-Step Cointegration Test

Step 1 - OLS regression. Estimate the linear relationship on training data:

$$P_{1,t} = \beta P_{2,t} + \alpha + \varepsilon_t$$

Step 2 - Augmented Dickey-Fuller test on residuals. Test the estimated error term for stationarity:

$$\Delta \varepsilon_t = \gamma \varepsilon_{t-1} + \sum_{i=1}^p \phi_i \Delta \varepsilon_{t-i} + e_t$$

- Null hypothesis (H_0):** $\gamma = 0$ — the residual spread has a unit root (non-stationary).

- **Selection rule:** accept the pair if the p-value < 0.05.

2. Dynamic Parameter Tracking via Kalman Filter

Static Ordinary Least Squares (OLS) over multi-year periods fails when underlying fundamentals shift. A state-space Kalman Filter updates β_t and α_t dynamically, bar by bar.

State-Space Formulation

State equation (unobserved hedge parameters):

$$\theta_t = [\alpha_t, \beta_t]^T = \theta_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0, W_t)$$

Observation equation (market price realization):

$$y_t = P_{1,t} = H_t \theta_t + v_t, \quad H_t = [1, P_{2,t}], \quad v_t \sim \mathcal{N}(0, V_t)$$

Per-Bar Update Equations

<i>Prior state prediction</i>	$\theta_{t t-1} = \theta_{t-1 t-1}$
-------------------------------	-------------------------------------

<i>Prior covariance</i>	$P_{t t-1} = P_{t-1 t-1} + W_t$
-------------------------	---------------------------------

<i>Innovation residual</i>	$e_t = y_t - H_t \theta_{t t-1}$
----------------------------	----------------------------------

<i>Innovation variance</i>	$Q_t = H_t P_{t t-1} H_t^T + V_t$
----------------------------	-----------------------------------

<i>Kalman gain</i>	$K_t = P_{t t-1} H_t^T Q_t^{-1}$
--------------------	----------------------------------

<i>Posterior state estimate</i>	$\theta_{t t} = \theta_{t t-1} + K_t e_t$
---------------------------------	---

<i>Posterior covariance</i>	$P_{t t} = P_{t t-1} - K_t H_t P_{t t-1}$
-----------------------------	---

3. The Ornstein-Uhlenbeck (OU) Process & Half-Life

Spread dynamics are modeled with a continuous-time stochastic Ornstein-Uhlenbeck process:

$$dS_t = \theta (\mu - S_t) dt + \sigma dW_t$$

where θ is the rate of mean reversion, μ is the equilibrium mean, σ is volatility, and dW_t is standard Brownian motion.

Half-Life of Mean Reversion (λ)

Fit an AR(1) regression on the discrete spread differences:

$$\Delta S_t = a + b S_{t-1} + \varepsilon_t$$

where $\theta = -\ln(1 + b)$. The half-life is then:

$$\lambda = \frac{\ln 2}{\theta} = \frac{-\ln 2}{\ln(1 + b)}$$

Optimal range: $5 \leq \lambda \leq 30$ bars.

4. Execution Logic & Signals

Dynamic Z-Score Signal Generation

$$S_t = P_{1,t} - (\beta_t P_{2,t} + \alpha_t)$$

$$Z_t = \frac{S_t - \text{Mean}_N(S)}{\text{StdDev}_N(S)}$$

Position Allocation

$$\text{Allocation per leg} = \frac{\text{Capital} \times 0.15}{2}$$

$$U_1 = \frac{\text{Allocation}}{P_{1,t}}, \quad U_2 = \frac{\text{Allocation} \times \beta_t}{P_{2,t}}$$

Triggers

Signal	Condition	Action
Long spread	$Z_t \leq -2.0$	BUY U_1 units of Asset 1, SHORT U_2 units of Asset 2
Short spread	$Z_t \geq +2.0$	SHORT U_1 units of Asset 1, BUY U_2 units of Asset 2
Take-profit exit	$Z_t = 0.0$	Close both legs when the spread fully converges to equilibrium
Stop-loss / circuit breaker	$ Z_t \geq 3.5$ or 7-day $p > 0.05$	Close both legs immediately to preserve capital if structural cointegration breaks down

5. Performance Comparison: Static OLS vs. Dynamic Kalman

Summary metrics across out-of-sample test datasets (\$100,000 baseline capital per pair):

Metric / Parameter	Static OLS Model	Dynamic Kalman + OU Model
Mean Half-Life	101.1 days	0.23 days
Average Universe Return	−4.79%	+0.15%
Worst Drawdown	−59.70% (AMD/NVDA)	−4.71% (AAPL/KLAC)
Top Win Rate	69.57% (MSFT/QCOM, +16.63% ret)	88.89% (MU/ORCL, +1.71% ret)

6. Full Python Implementation Script

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from statsmodels.tsa.stattools import coint
import yfinance as yf

class KalmanFilterHedgeRatio:
    def __init__(self, delta=1e-4, R=1e-3):
        self.delta = delta
        self.R = R
        self.Vw = (self.delta / (1 - self.delta)) * np.eye(2)
        self.Ve = self.R
        self.theta = np.zeros(2) # [alpha, beta]
        self.P = np.zeros((2, 2))

    def update(self, x, y):
        P_prior = self.P + self.Vw
        H = np.array([1.0, x])
        y_hat = np.dot(H, self.theta)
        error = y - y_hat
        Qt = np.dot(H, np.dot(P_prior, H.T)) + self.Ve
        K = np.dot(P_prior, H.T) / Qt
        self.theta = self.theta + K * error
        self.P = P_prior - np.outer(K, np.dot(H, P_prior))
        return self.theta[0], self.theta[1], error

def estimate_ou_parameters(spread_series):
    if len(spread_series) < 30:
        return np.nan, np.nan, np.nan
    s_lag = spread_series.shift(1).dropna()
    s_curr = spread_series.iloc[1:]
    X = sm.add_constant(s_lag)
    model = sm.OLS(s_curr, X).fit()
    a, b = model.params.iloc[0], model.params.iloc[1]
    if b >= 1.0 or b <= 0.0:
        return np.nan, np.nan, np.nan
    theta = -np.log(b)
    mu = a / (1.0 - b)
```

```
half_life = np.log(2.0) / theta
return half_life, mu, theta
```

```
class StatArbEngine:
```

```
def __init__(self, initial_capital=100000.0, z_entry=2.0, z_exit=0.0, z_stop=3.5):
    self.capital = initial_capital
    self.z_entry = z_entry
    self.z_exit = z_exit
    self.z_stop = z_stop
```

```
def run(self, s1_series, s2_series, lookback=252, rescreen_days=7):
    kf = KalmanFilterHedgeRatio()
    equity_curve = [self.capital]
    trades = []
    in_pos = False
    pos_type = None
    entry_p1, entry_p2, u1, u2 = 0.0, 0.0, 0.0, 0.0
    spread_history = []
    is_cointegrated = True
```

```
    for i in range(len(s1_series)):
        p1, p2 = s1_series.iloc[i], s2_series.iloc[i]
        alpha_t, beta_t, err_t = kf.update(p2, p1)
        spread_t = p1 - (beta_t * p2 + alpha_t)
        spread_history.append(spread_t)

        if i % rescreen_days == 0 and i >= lookback:
            _, pval, _ = coint(s1_series.iloc[i-lookback:i],
                               s2_series.iloc[i-lookback:i])
            is_cointegrated = (pval < 0.05)

        if len(spread_history) >= 60:
            rec = pd.Series(spread_history[-60:])
            std_sp = rec.std()
            z_t = (spread_t - rec.mean()) / std_sp if std_sp > 0 else 0.0
        else:
            z_t = 0.0

        if not in_pos:
```

```
            if is_cointegrated and abs(z_t) >= self.z_entry:
                pos_type = 'LONG_SPREAD' if z_t <= -self.z_entry else 'SHORT_SPREAD'
                in_pos = True
                alloc = self.capital * 0.15
                u1 = (alloc / 2.0) / p1
                u2 = ((alloc / 2.0) * beta_t) / p2
                entry_p1, entry_p2 = p1, p2
```

```
            elif in_pos:
                tp = (pos_type == 'LONG_SPREAD' and z_t >= self.z_exit) or \
                    (pos_type == 'SHORT_SPREAD' and z_t <= self.z_exit)
                sl = (abs(z_t) >= self.z_stop) or (not is_cointegrated)
```

```
            if tp or sl:
                if pos_type == 'LONG_SPREAD':
                    pnl = (u1 * (p1 - entry_p1)) + (u2 * (entry_p2 - p2))
                else:
                    pnl = (u1 * (entry_p1 - p1)) + (u2 * (p2 - entry_p2))
                self.capital += pnl
                equity_curve.append(self.capital)
                trades.append({'pnl': pnl, 'capital': self.capital})
                in_pos = False
```

```
    return pd.DataFrame(trades), np.array(equity_curve)
```

7. Operational Pre-Flight Checklist

Item	Requirement
Broker Mechanics	Ensure your brokerage account supports borrowing and short-selling for the hedge asset leg.
Execution Window	Submit orders 5 to 10 minutes before market close (daily strategies), or use 1-hour / 15-minute intraday bars with limit orders.
Paper Trading Phase	Run the system in a simulated environment for 10 consecutive trading sessions to verify execution alignment against backtest parameters.